

okDriver Smart Dashcams Private Limited

FULL STACK DEVELOPER HIRING

Integrated CCTV Monitoring, Video Analytics and Real-Time Alert Platform

1. About okDriver

okDriver Smart Dashcams Private Limited is building an AI-powered road safety and mobility intelligence platform combining dashcams, Computer Vision, IoT devices, real-time video, driver analytics, emergency intelligence, and operational dashboards.

For this hiring challenge, we want to evaluate your ability to build the software layer connecting live video sources, APIs, AI events, databases, alerts, maps, and operators into one reliable platform.

2. Challenge Objective

Build a working prototype of a centralized CCTV monitoring and video analytics platform. Multiple camera sources should be onboarded into one system, monitored through a unified interface, enriched with metadata and AI events, correlated against a representative watchlist, and used to generate real-time alerts.

The challenge is aligned with the Gujarat Police Innovation Hackathon 2026, whose official problem statement focuses on heterogeneous CCTV infrastructure, unified analytics, interoperability, security, watchlist correlation, GIS, real-time alerts, and scalability toward approximately 80,000 cameras. .

3. Camera Registry and Onboarding

Create a camera registry where an administrator can add, edit, disable, search, and filter camera sources. Store camera ID, name, department, latitude, longitude, camera type, source protocol, stream endpoint reference, status, last heartbeat, zone, and relevant storage metadata.

- Support manual onboarding and API-based onboarding.
- Show camera health as Online, Offline, or Degraded.
- Provide search, filters, and basic audit history.
- Show camera locations on an interactive map.

Example: Camera C001 may represent a traffic junction in Ahmedabad and C002 an RTO checkpoint. Both should be visible in the same registry and map.

4. Live Video Integration

Integrate at least two logically different camera sources. You may use legally usable sample CCTV footage, your own recorded feeds, test streams, or simulated sources. If real RTSP or ONVIF cameras are unavailable, create a documented adapter or simulator so the architecture can later accept RTSP, ONVIF, vendor APIs, or SDKs.

The official challenge identifies RTSP, ONVIF, vendor SDKs and APIs, with WebRTC or HLS relay approaches, as relevant integration mechanisms.

5. Unified Monitoring Dashboard

Build an operational dashboard, not a basic CRUD page. An operator should understand system health, active events, cameras, alerts, and recent activity from one screen.

- Live or near-live camera grid
- Online/offline camera count
- Active alerts and recent detections
- Camera search and filtering
- Vehicle/entity search
- Timestamped event history
- Alert severity and status
- Camera health indicators
- System statistics

6. GIS and Movement Visualization

Add an interactive map using Leaflet, OpenLayers, Mapbox, Google Maps, or an equivalent technology. Camera markers should reflect status and alert state.

The platform should also plot a vehicle's detected movement history when multiple camera events are available.

Example: If GJ01XX0001 is detected at Camera A at 10:02, Camera B at 10:18, and Camera C at 10:41, display the chronological history and available route on the map. Vehicle tracing is part of the official technical evaluation scenario. .

7. AI Video Analytics Integration

You do not need to train the AI model for this full stack challenge. Instead, build the software interface that receives AI inference results and converts them into useful operational events.

Implement at least one analytics event such as ANPR, vehicle detection, person detection, or object detection.

Example event: camera_id, timestamp, vehicle_number, confidence, vehicle_type, bounding_box, event_type. The backend should validate and store it, correlate it with the watchlist, and update the dashboard in real time.

The official problem statement includes ANPR, facial recognition, object detection, person and vehicle tracking, and other intelligent analytics. .

8. Watchlist and Real-Time Alerting

Create a representative watchlist containing synthetic or legally usable records such as stolen vehicles, blacklisted vehicles, wanted persons, missing persons, or other entities of interest.

- Add and manage watchlist records.
- Receive analytics events through API or message channels.
- Match detected identifiers against the watchlist.
- Generate an alert immediately when a match occurs.

- Store timestamp, camera, location, matched entity, confidence, and status.
- Allow operators to acknowledge or resolve alerts.

Example: A detected vehicle number matches a blacklisted vehicle. The operator immediately receives an alert showing the vehicle number, camera, location, timestamp, confidence, and relevant feed or event.

9. Real-Time Data Architecture

Use WebSocket, Server-Sent Events, Redis Pub/Sub, Kafka, RabbitMQ, or another appropriate mechanism to demonstrate real-time synchronization.

- New detection events appear without page refresh.
- Watchlist matches push alerts immediately.
- Camera health changes update the dashboard.
- Events remain persisted for search and reporting.
- The design should prevent duplicate or uncontrolled event generation.

10. Suggested Technology Direction

okDriver currently works with FastAPI, Python, Node.js, Express, RESTful APIs, MySQL, RDS, WebRTC and WebSocket. Candidates may choose equivalent technologies where justified, but should demonstrate that they can work with a production-style API and real-time architecture.

- Backend: FastAPI/Python and/or Node.js/Express
- Database: MySQL or PostgreSQL, with RDS-compatible design preferred
- Real-time: WebSocket, WebRTC, Redis, Kafka or RabbitMQ
- Frontend: React, Next.js, Vue or another professional framework
- Maps: Leaflet, OpenLayers, Mapbox or equivalent
- Video: RTSP, WebRTC, HLS or adapter-based architecture
- Deployment: Docker preferred
- API documentation: OpenAPI/Swagger preferred

Do not submit a superficial MERN template. We will evaluate system architecture, video integration, API design, real-time synchronization, database modelling, security, and scalability.

11. Minimum End-to-End Flow

- Admin adds two or more camera sources.
- Camera metadata appears in the registry and map.
- Camera status is monitored.
- A live or recorded feed is viewable.
- An analytics service produces a detection event.
- Backend validates and stores the event.
- The event is checked against the watchlist.
- A matching record generates a real-time alert.
- The operator searches the entity and sees movement history.
- The system maintains an auditable history of events and alerts.

12. Scalability and Production Thinking

You do not need to actually operate 80,000 cameras. You must explain how your architecture can evolve from a prototype to a large distributed platform.

- Central, regional and edge processing
- Horizontal scaling and load balancing
- Video bandwidth and low-bandwidth strategies
- Database indexing and scaling
- Caching and message queues
- GPU/accelerator requirements
- Hot, warm and cold storage
- Monitoring, logging and health checks
- High availability and disaster recovery
- Cybersecurity and network segmentation
- Estimated infrastructure and operational costs

The official hackathon specifically asks participants to address central, regional and edge compute, GPU requirements, bandwidth, storage, load balancing, monitoring, disaster recovery, cybersecurity, and costs. .

13. Security Requirements

- Authentication and role-based authorization
- Input validation and secure API design
- Secure handling of stream URLs and credentials
- HTTPS/TLS for exposed services
- Audit logs for important actions
- Rate limiting where appropriate
- No secrets or API keys committed to GitHub
- Environment-based configuration

14. Required Deliverables

This must be a working prototype. Concept-only screens, animations, or a video without an operational backend are not sufficient. The official hackathon similarly requires a fully functional own-feed demonstration. .

- GitHub repository with source code.
- README with setup, architecture, environment variables, database setup, APIs and known limitations.
- Architecture diagram covering sources, ingestion, backend, database, AI events, watchlist, alerts, dashboard and map.
- Working dashboard demonstration.
- Representative camera/video feed.
- Representative watchlist dataset.
- API documentation.
- Database schema or ER diagram.
- Short screen-recorded demo.
- Scalability and deployment note.

15. Short Video Requirement

Record a 3 to 5 minute video showing the actual working system. Focus on demonstration rather than slides.

- Camera onboarding
- Dashboard and map
- Live or near-live feed
- AI detection event
- Watchlist match
- Real-time alert
- Entity search and movement history
- One API/backend workflow
- Brief architecture and scalability explanation

16. Evaluation Criteria

Area	Evaluation Focus	Priority
Working product	End-to-end functionality	Very High
Architecture	Modular, scalable, maintainable design	Very High
Real-time engineering	Event flow, synchronization, reliability	High
Video integration	Practical camera/stream handling	High
API and database	Schema, validation, indexing, clean APIs	High
Dashboard	Operational clarity and visualization	High
Watchlist and alerts	Matching and real-time workflow	High
Scalability	Path toward large deployment	High
Security	Authentication, secrets, data protection	High
Code quality	Structure, documentation and Git hygiene	Medium

17. Expected Outcome

The completed prototype should demonstrate that you can build a small but functional version of a real video intelligence platform.

- Users can onboard and manage cameras.
- Multiple sources can be represented in one platform.
- Video can be viewed from a unified interface.
- AI events can enter through APIs or event streams.
- Events can be stored, searched and correlated.
- Watchlist matches can trigger real-time alerts.
- Vehicles or entities can be traced across camera events.
- Camera locations and movement history can be visualized.
- The architecture has a credible path to large-scale deployment.
- Another engineer can understand and continue the codebase.

18. Optional Advanced Features

- ONVIF discovery or mock adapter
- RTSP to WebRTC/HLS gateway
- Cross-camera vehicle re-identification
- ANPR duplicate suppression
- Event deduplication and temporal correlation
- Camera heartbeat service
- Role-based operator/admin views
- Route reconstruction
- Low-bandwidth edge gateway
- Docker Compose
- Automated tests and CI/CD
- Prometheus/Grafana or equivalent observability
- Redis/Kafka event architecture
- Multi-department tenancy

19. Official Hackathon Reference

Review the official Gujarat Police Innovation Hackathon 2026 problem statement before beginning. It defines reference models including CCTV Registry and GIS Foundation, Unified Viewing and Metadata Analytics, VMS Federation and Middleware, Central VMS, and Hybrid or Innovative Architecture. It also specifies requirements around heterogeneous CCTV integration, AI analytics, watchlist correlation, alerts, government-feed demonstrations, scalability, security, infrastructure sizing, and deployment planning. .

Official reference: <https://sentinel.gujarat.gov.in/problems>

20. Submission Instructions

Once completed, email the complete submission to experts@okdriver.in

- Subject: Full Stack Developer Hackathon Submission | <Candidate Name>
- Candidate name and contact number
- Role applied for
- GitHub repository link
- Live hosted platform link, if available
- 3 to 5 minute video link, preferably unlisted YouTube or accessible Drive/OneDrive
- Architecture document or PDF
- Demo credentials, if required
- Short note on completed work, limitations, and improvements you would make with more time

Make sure all links are accessible to the review team. Never commit passwords, private keys, API keys, or production credentials to the repository.